PROJECT REPORT

ON

ONLINE QUIZ APPLICATION

(SkillForge LMS – Examination & Quiz Portal)

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Arnav Mittal (24BCS10123)
Rajan (24BCS12890)
Class / Section: 24BCS_VT_BD_710 – B

Harish Mittal
Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHANDIGARH UNIVERSITY, MOHALI, PUNJAB

NOV 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “ONLINE QUIZ APPLICATION” is a bonafide work carried out by the following students of the Department of Computer Science and Engineering, Chandigarh University, in partial fulfilment of the requirements for the award of the Bachelor of Engineering degree in Computer Science and Engineering.

Arnav Mittal (24BCS10123)
Rajan (24BCS12890)

The work presented in this report is based on the project developed by the students. To the best of our knowledge, the work has been completed under the guidance and supervision of the undersigned.

_____

Professor Harish Mittal
SUPERVISOR
Professor
Department of Computer Science and Engineering
Chandigarh University, Mohali, Punjab

_____

Gagandeep Singh
Head of Department
Department of Computer Science and Engineering
Chandigarh University

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Chandigarh University and the Department of Computer Science and Engineering for providing us with the opportunity and resources to complete the project titled “Online Quiz Application”.

We are thankful to our project supervisor for valuable guidance, encouragement and suggestions throughout the development of this project. The feedback received during the planning, implementation and documentation stages helped us understand the importance of systematic software development.

We also acknowledge the support of our faculty members and classmates who provided useful suggestions and helped us improve our understanding of Java programming, Servlets, JSP, XML processing and web application design.

We are grateful to the developers and maintainers of the technologies used in this project, including Java, Jakarta Servlet, JavaServer Pages, JSTL, Apache Tomcat, Apache Maven, HTML, CSS and JavaScript. Their documentation and learning resources were useful during implementation.

Finally, we thank our families for their patience, motivation and support. This project has helped us connect theoretical concepts with practical implementation and has improved our problem-solving and technical communication skills.

Arnav Mittal (24BCS10123)
Rajan (24BCS12890)

TABLE OF CONTENTS

Section	Page
Title Page	i
Bonafide Certificate	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures and Tables	v
Abstract	vi
Graphical Abstract	vii
Abbreviations and Symbols	viii
Chapter 1: Introduction	8
1.1 Background and Need	8
1.2 Problem Identification and Task Identification	8
1.3 Objectives and Scope	9
1.4 Timeline and Report Organisation	10
Chapter 2: Literature Survey	11
2.1 Background and Existing Approaches	11
2.2 Proposed Solution and Project Goals	12
Chapter 3: Design Flow / Process	14
3.1 Requirements and Concept Generation	14
3.2 Architecture and Data Design	15
3.3 Algorithms and Implementation Plan	17
Chapter 4: Results Analysis and Validation	20
4.1 Implementation and User Interface	20
4.2 Testing and Validation	24
Chapter 5: Conclusion and Future Work	26
References	27
Appendix A: User Manual	28
Appendix B: Project Structure and Achievements	30

LIST OF FIGURES AND TABLES

List of Figures

Figure 1.1: High-level view of the online examination problem
Figure 3.1: Three-layer application architecture
Figure 3.2: Quiz attempt and evaluation flow
Figure 3.3: XML data model overview
Figure 4.1: Login page
Figure 4.2: Student assessment center (dashboard)
Figure 4.3: Online assessment session
Figure 4.4: Examination grade report
Figure 4.5: Instructor administration console
Figure 4.6: Define new assessment form

List of Tables

Table 1.1: Users of the system
Table 1.2: Project objectives
Table 1.3: Development timeline
Table 2.1: Comparison of examination approaches
Table 3.1: Functional requirements
Table 3.2: Technology stack
Table 3.3: Main backend packages
Table 3.4: XML data stores
Table 3.5: URL mapping of servlets
Table 4.1: Pre-configured accounts
Table 4.2: Test cases and outcomes

ABSTRACT

The Online Quiz Application (branded in the interface as “SkillForge LMS – Institutional Examination & Quiz Portal”) is a web-based system developed to conduct and evaluate multiple-choice examinations over a browser. It provides two role-based portals. Instructors (administrators) can create, edit, publish and delete quizzes, manage the questions inside each quiz, review all student submissions and manage user accounts. Students can register, view published quizzes, attempt a timed exam, receive an instantly evaluated grade report with explanations, and review their attempt history.

The application is implemented with Java Servlets and JavaServer Pages (JSP) using the Jakarta EE 9 (Servlet 5.0) API, packaged as a WAR file with Apache Maven and deployed on Apache Tomcat 10.1. Instead of a relational database, data is stored in three XML files (users.xml, quizzes.xml and attempts.xml) that are read and written through the Java DOM and Transformer APIs, so the project runs without any database setup. A servlet filter (AuthFilter) protects the /admin/* and /student/* areas by role. A JavaScript controller provides a live countdown timer, question navigation and automatic submission when time expires. The report explains the problem, design decisions, data model, implementation, validation and the steps to run the project.

GRAPHICAL ABSTRACT

ONLINE QUIZ APPLICATION

STUDENT / INSTRUCTOR (ADMIN)
Login → Role Check → Dashboard
↓
TOMCAT 10.1 : SERVLETS + JSP
AuthFilter → Servlets → XMLHelper (DOM / Transformer)
↓
XML DATA STORE (WEB-INF/data)
users.xml | quizzes.xml | attempts.xml
↓
RESULT
Automatic marking → Pass / Fail grade report → Admin review

System Summary

The user opens the application in a browser and signs in. The AuthFilter checks the session and role and sends the user to the student or admin dashboard. Servlets receive HTTP requests, call the XMLHelper utility class to read or update the XML files, and forward the data to JSP pages that render the HTML response. When a student submits a quiz, the servlet compares each selected option with the stored correct option, calculates marks and percentage, saves the attempt, and redirects to the grade report.

ABBREVIATIONS AND SYMBOLS

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JSP	JavaServer Pages
JSTL	JSP Standard Tag Library
JVM	Java Virtual Machine
LMS	Learning Management System
MCQ	Multiple Choice Question
MVC	Model-View-Controller
UI	User Interface
URL	Uniform Resource Locator
WAR	Web Application Archive
XML	Extensible Markup Language

CHAPTER 1
INTRODUCTION

1.1 Background

Examinations and short quizzes are a regular part of academic learning. When they are conducted on paper, an instructor must prepare question papers, supervise the session, check every answer sheet manually and then record the marks. This is slow, can introduce marking errors, and makes it difficult to keep a searchable history of how each student has performed over time.

The Online Quiz Application developed in this project is a small web-based examination portal. It shows how question setting, timed examination, automatic marking and result review can be handled in one application. Instructors prepare quizzes and publish them; students sign in, take the quiz within the allotted time and immediately see their score, the correct answers and the explanation for each question. A default administrator account, a demo student account and a sample “Java Basics” quiz are provided when the application starts for the first time.

1.2 Identification of Need

The need for this project is to demonstrate a centralised digital method for creating and conducting quizzes. The application removes manual answer-sheet checking, applies the same marking rule to every student, and keeps a record of every attempt that an instructor can inspect later. It also demonstrates how a complete role-based web application can be built using core Java web technologies without requiring a database server.

1.3 Relevant Contemporary Issues

- Reducing the time and effort spent on manually evaluating objective-type answers.
- Giving students immediate feedback together with an explanation of the correct answer.
- Enforcing a time limit so that every candidate gets the same duration.
- Restricting administrative functions to authorised instructors only.
- Keeping quiz content, user accounts and results in a structured and portable format.
- Making the interface usable on different screen sizes.

1.4 Project Context

The supplied implementation is a local web application deployed on Apache Tomcat 10.1.61 and accessed at http://localhost:8080/JavaQuizApp/. It stores its data in XML files inside the WEB-INF/data folder of the deployed application, which makes it easy to run on any machine with a Java runtime. The project is an academic simulation; it is not connected to any institutional student-information system or to an external identity provider.

1.5 Problem Identification

The main problem addressed is the conduct of objective examinations with limited time and the fair, automatic evaluation of the answers. The system must decide, for each question, whether the selected option matches the correct option, award the configured marks, compute the percentage, compare it with the passing benchmark and preserve the result so that both the student and the instructor can review it later.

1.6 Task Identification

1. Design the application structure and select suitable technologies.
2. Define model classes for users, quizzes, questions, options and quiz attempts.

3. Implement registration, login, logout and role-based access control.
4. Implement quiz management (create, edit, delete, publish or unpublish) for the instructor.
5. Implement question management with four options, marks, per-question time and an explanation.
6. Implement the student dashboard that lists published quizzes with summary statistics.
7. Implement the timed quiz session with question navigation and automatic submission.
8. Implement automatic evaluation, grade report and attempt history.
9. Implement the administrator views for submissions, filtering and user management.
10. Prepare the user interface, XML data store and instructions for building and running the project.

1.7 Identification of Users

User type	Main activities
Student	Register, log in, view published quizzes, attempt quizzes, view grade reports and attempt history.
Instructor / Administrator	Create and publish quizzes, manage questions, review all student submissions and manage user accounts.
Project evaluator	Inspect the workflow, source code, data files and deployed behaviour of the application.

Table 1.1: Users of the system

1.8 Project Boundaries

The project focuses on multiple-choice quizzes with a single correct option per question. Question types such as descriptive answers, code submission or file upload, proctoring or anti-cheating measures, e-mail notifications, integration with a learning-management platform and production-grade multi-user data storage are outside the supplied implementation. These items are considered possible future enhancements.

1.9 Goals and Objectives

Objective	Description
Authentication	Allow users to register, log in and log out, with separate student and administrator roles.
Quiz management	Allow the instructor to create, edit, delete and publish quizzes with time limit and passing score.
Question management	Add, edit and delete questions with four options, marks and an explanation.
Timed examination	Provide a live countdown timer, question navigator and automatic submission at timeout.
Automatic evaluation	Mark each answer, compute percentage and decide pass or fail.
Result review	Show correct, incorrect and unanswered counts with a question-wise explanation.
Administration	Show statistics, recent submissions and registered users to the instructor.

Table 1.2: Project objectives

1.10 Scope and Significance

The project is useful as an academic demonstration of web application development using core Java technologies. It combines object-oriented design, the Servlet and JSP request-response cycle, session handling, filters, XML parsing with the DOM API, and client-side JavaScript. It also shows how presentation (JSP), request handling (servlets), business rules and data access (XMLHelper) can be separated.

1.11 Development Timeline

Phase	Activities	Deliverable
Week 1	Requirement understanding, feature selection and study of Servlet/JSP and XML processing	Requirement list and initial design
Week 2	Model classes, XML data files, XMLHelper and authentication with AuthFilter	Data layer and login flow
Week 3	Admin quiz and question management, student dashboard and quiz session with timer	Working examination workflow
Week 4	Result analysis, admin submissions, styling, testing and report preparation	Integrated project and documentation

Table 1.3: Development timeline

The timeline is presented as a project-planning summary. Actual dates may be edited by the student if required by the department.

1.12 Organisation of the Report

Chapter 1 introduces the project, its need, problem statement, objectives, scope and timeline. Chapter 2 discusses the software concepts relevant to this implementation. Chapter 3 explains the requirements, architecture, data design, alternatives, constraints and algorithms. Chapter 4 describes the implemented modules with screenshots and the validation approach. Chapter 5 presents the conclusion, limitations and future work. The appendices provide the user manual, project structure and supplementary information.

1.13 Summary

The Online Quiz Application addresses a clearly defined academic problem: creating, conducting and automatically evaluating timed multiple-choice quizzes in a single web application. The remaining chapters explain how the selected technologies and layered design are used to implement these functions.

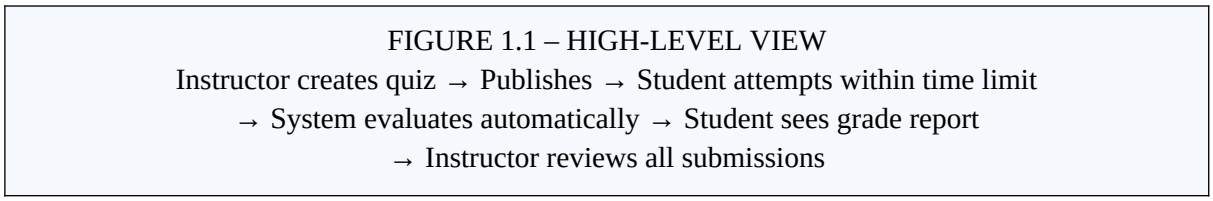


Figure 1.1: High-level view of the online examination problem

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

A literature survey helps identify established ideas and technologies related to the project. This report focuses on the software engineering concepts directly used in the supplied code: web-based information systems, the Servlet and JSP model, session-based authentication with filters, XML as a data format, and client-side scripting for timed interaction.

2.2 Development of Online Examination Systems

Examinations were traditionally conducted with printed question papers and hand-evaluated answer sheets. As computers and the web became widely available, institutions began to use online testing, in which questions are stored in a database, presented through a browser and marked automatically. Modern learning management systems commonly provide quiz modules with timers, question banks, instant feedback and result analytics.

This project follows the same general pattern at a smaller academic scale. It does not claim to reproduce the scale, security measures or integrations of a commercial learning-management platform.

2.3 Web Application Architecture

A web application commonly separates presentation, application processing and data storage. The presentation layer accepts user input and displays results. The application layer validates requests and applies business rules. The data layer stores and retrieves records. This separation improves maintainability because a change in one layer does not necessarily require a complete rewrite of the others.

2.4 Java Servlets and JSP

A servlet is a Java class that runs inside a servlet container and handles HTTP requests and responses. JavaServer Pages allow HTML to be mixed with Java-based tags so that dynamic pages can be produced. In this project, servlets are mapped to URLs using the @WebServlet annotation, process the request and forward the prepared data to JSP files under WEB-INF/jsp using a RequestDispatcher. Keeping the JSP files inside WEB-INF prevents users from opening them directly by URL.

2.5 Servlet Filters and Sessions

A filter intercepts requests before they reach a servlet. The project uses a filter mapped to /admin/* and /student/* to check whether a user is stored in the HTTP session and whether the role matches the requested area. HttpSession is also used to remember the quiz start time so that the time taken can be calculated on the server when the quiz is submitted.

2.6 XML as a Data Store

XML is a text-based format that represents data as nested elements and attributes. The Java DOM API loads an XML document into memory as a tree that can be searched and modified, and the Transformer API writes the tree back to a file. For a small application, XML files remove the need for a separate database server and are easy to inspect and back up. Their limitations are the lack of indexing, transactions and efficient concurrent access.

2.7 Apache Maven and Tomcat

Apache Maven manages project metadata, dependencies and packaging; the project uses the war packaging type to produce JavaQuizApp.war. Apache Tomcat is a servlet container that implements Jakarta Servlet and JSP specifications. Tomcat 10.1 supports the jakarta.* namespace that the code uses.

2.8 Client-side Timer and Navigation

Because the countdown must update every second without reloading the page, the project uses JavaScript. A controller object keeps the current question index and remaining seconds, updates the progress bar and question navigator, warns the student when less than one minute remains and submits the form automatically when time reaches zero.

2.9 Comparison of Approaches

Approach	Advantages	Limitations
Paper-based examination	No software needed; familiar to everyone.	Manual checking is slow and error-prone; results are hard to search.
Single-file program with mixed HTML and logic	Fast to start for a tiny demonstration.	Presentation, logic and data code become tightly coupled.
Layered Servlet/JSP application with XML storage	Clear separation of filter, servlets, JSP pages and data helper; no database installation required.	Requires a servlet container; XML storage does not scale to many concurrent users.

Table 2.1: Comparison of examination approaches

2.10 Proposed Solution

The proposed solution is a layered Java web application. A filter guards role-based areas, servlets receive requests and apply business rules, an XMLHelper class reads and writes the XML data, model classes represent the data, and JSP pages produce the browser interface. A JavaScript controller supports the timed examination session.

2.11 Problem Definition

Design and implement a web-based quiz application that maintains users, quizzes and questions, supports role-based login, lets instructors publish quizzes, lets students attempt them within a time limit, evaluates answers automatically and preserves every attempt so that results can be reviewed by the student and the instructor.

2.12 Project Goals

- Provide separate, secured portals for students and instructors.
- Maintain structured quiz, question and result data in XML files.
- Automate marking, percentage calculation and pass/fail decision.
- Give students a clear explanation for every question after submission.
- Demonstrate a maintainable layered architecture.

2.13 Project Objectives and Expected Outcomes

The expected outcome is a deployable WAR file that runs on Tomcat. It should be possible to sign in as administrator or student, view the sample quiz, attempt it, receive a grade report, and

see the submission appear in the administrator console. The dashboards should display statistics computed from the stored data rather than fixed values.

2.14 Limitations of the Survey

This survey is a technology-oriented review based on the implementation and standard software engineering concepts. The project ZIP does not contain a formal bibliometric dataset or a collection of research papers with citation counts. Therefore, no quantitative bibliometric analysis or global publication trend is claimed.

CHAPTER 3
DESIGN FLOW / PROCESS

3.1 Requirement Analysis

Requirement analysis identifies what the application must do and the quality conditions it should satisfy. The requirements below are derived from the features and structure of the supplied project.

3.2 Functional Requirements

ID	Requirement
FR-01	The system shall allow a new student to register with a unique username.
FR-02	The system shall authenticate a user and support logout.
FR-03	The system shall restrict /admin/* pages to administrators and /student/* pages to students.
FR-04	The administrator shall be able to create, edit and delete quizzes.
FR-05	The administrator shall be able to publish or unpublish a quiz.
FR-06	The administrator shall be able to add, edit and delete questions with four options and an explanation.
FR-07	The student dashboard shall list only published quizzes with completed, passed, average and highest statistics.
FR-08	The system shall present a quiz one question at a time with a countdown timer.
FR-09	The system shall submit the quiz automatically when the timer expires.
FR-10	The system shall evaluate answers, calculate marks and percentage and mark the attempt as passed or failed.
FR-11	The system shall display a grade report and keep the attempt history.
FR-12	The administrator shall view all submissions, filter them by quiz and open a detailed evaluation.
FR-13	The administrator shall view and delete user accounts (except the logged-in administrator).

Table 3.1: Functional requirements

3.3 Non-Functional Requirements

- Usability: navigation and forms should be understandable to a typical user.
- Maintainability: responsibilities should be separated into packages and classes.
- Portability: the project should run on any system with a compatible JDK and Tomcat; no database installation is needed.
- Data persistence: results and quizzes should survive application restarts through the XML files.
- Performance: routine operations should complete within a reasonable time on a local machine.

3.4 Concept Generation and Alternatives

Criteria	Alternative A: Single-page script with embedded data	Alternative B: Layered Servlet/JSP with XML store
Structure	Interface, logic and data closely connected.	Filter, servlets, JSP pages, models and data helper are separated.
Maintainability	Changes may affect several parts of the program.	Responsibilities are easier to locate and modify.
Data persistence	Data lost or hard-coded in the page.	Quizzes and attempts persist in XML files.
Security	Role separation is difficult.	AuthFilter enforces roles in one place.
Suitability	Suitable for a very small demonstration.	More suitable for a multi-feature examination workflow.

3.5 Selected Design

Alternative B is selected because the project contains several related modules such as authentication, quiz management, question management, quiz attempt, evaluation and submissions review. A layered structure makes the code easier to organise and supports future changes, for example replacing the XML store with a relational database by changing only the data-access class.

3.6 Technology Stack

Layer / Purpose	Technology used
Programming language	Java (compiler level 11)
Web API	Jakarta Servlet 5.0 and JSP 3.0
View helpers	JSTL 2.0
Data storage	XML files (users.xml, quizzes.xml, attempts.xml) via Java DOM and Transformer
Build and packaging	Apache Maven, WAR packaging (also build.bat and build.ps1 scripts)
Server	Apache Tomcat 10.1.61
Frontend	HTML, CSS (style.css) and JavaScript (quiz-timer.js)
Application URL	http://localhost:8080/JavaQuizApp/

Table 3.2: Technology stack

3.7 Design Constraints

The implementation is configured for a local Tomcat server. Data files are located inside the deployed WEB-INF/data folder, so the data lives with the deployed copy of the application. The project uses a single correct option per question and is not designed for a large number of simultaneous users.

3.8 System Architecture

The system follows a layered architecture. The browser sends HTTP requests to Tomcat. The AuthFilter checks the session and role. Servlets map requests to actions, call XMLHelper to

read or write data and forward the result to JSP pages. Model classes carry the data between layers.

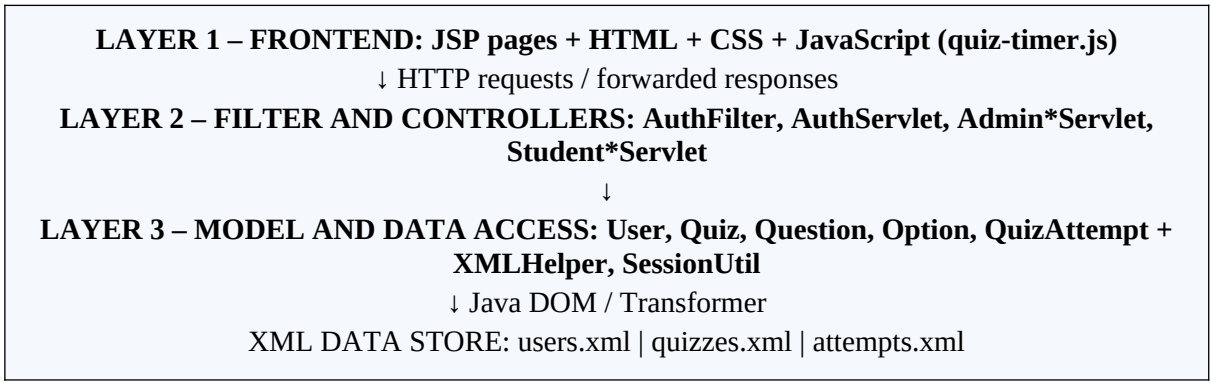


Figure 3.1: Three-layer application architecture

3.9 Main Backend Packages

Package (com.quiz.*)	Responsibility
filter	AuthFilter – role-based guard for /admin/* and /student/*.
model	User, Quiz, Question, Option and QuizAttempt classes.
servlet	AuthServlet, AdminDashboardServlet, AdminQuizServlet, AdminQuestionServlet, AdminAttemptServlet, AdminUserServlet, StudentDashboardServlet, StudentQuizServlet and StudentHistoryServlet.
util	XMLHelper (all XML reading/writing), SessionUtil (session and data-path helpers) and AppContextListener (initialises data files at startup).

Table 3.3: Main backend packages

3.10 Data Design

Instead of relational tables, the application uses three XML documents. Each document has a root element that contains repeated child elements, one per record. Quiz questions and options are nested inside their quiz element, and the attempt document stores one element per attempt with the individual question results embedded in it.

XML file	Important elements	Purpose
users.xml	id, username, password, email, role (ADMIN/STUDENT), fullName, createdAt	Account and role records.
quizzes.xml	id, title, description, category, totalTimeLimit, timePerQuestion, published, createdBy, createdAt, passingScore, questions (id, text, type, marks, timeLimit, options with id and correct flag, explanation)	Quizzes with embedded questions.
attempts.xml	id, quizId, quizTitle, username, studentName, startedAt, completedAt, timeTaken, totalMarks, obtainedMarks, percentage, passed, per-question selected/correct option and marks	Student submission records.

Table 3.4: XML data stores

FIGURE 3.3 – XML DATA MODEL
users.xml (user) ← username → attempts.xml (attempt)
quizzes.xml (quiz → questions → options) ← quizId → attempts.xml
Admin manages users and quizzes; student creates attempts

Figure 3.3: XML data model overview

3.11 Relationships and Constraints

- An attempt references a quiz through quizId and a student through username.
- A quiz contains many questions, and each question contains four options with exactly one marked correct.
- Username is checked for uniqueness at registration.
- Only quizzes with published = true appear on the student dashboard.
- The logged-in administrator cannot delete their own account.

3.12 Initial Data

AppContextListener runs when the application starts. If a data file is missing or empty in WEB-INF/data, it copies the default file from the application resources or writes built-in default content. The defaults contain an administrator account, one student account (John Doe) and a published quiz “Java Basics” with three questions worth ten marks each.

3.13 Authentication and Access Control

AuthServlet handles /login, /register and /logout. On login, XMLHelper.authenticateUser compares the supplied credentials with users.xml; on success the User object is stored in the HttpSession and the user is redirected to the proper dashboard. AuthFilter then checks every request to /admin/* and /student/*: a visitor without a session is sent to the login page, an administrator opening a student page is redirected to the admin dashboard, and a student opening an admin page is redirected to the student dashboard. The session cookie is configured as HttpOnly with a 60-minute session timeout in web.xml.

3.14 Quiz Attempt and Evaluation Algorithm

When the student opens /student/quiz/start, the servlet checks that the quiz exists, is published and has questions. It stores the start time in the session and forwards to the quiz page. On submission, the servlet evaluates the answers as follows.

Algorithm: Quiz Evaluation

1. Receive the quiz id and the selected option for each question from the submitted form.
2. Load the quiz from quizzes.xml; calculate time taken from the start time stored in the session.
3. For every question, add its marks to the total marks.
4. Compare the selected option with the stored correct option (case-insensitive). If they match, add the question’s marks to the obtained marks; otherwise award zero.
5. Record the selected option, correct option and marks for each question in a QuestionAttempt object.
6. Calculate percentage = obtained marks ÷ total marks × 100, rounded to one decimal place.

7. Mark the attempt as passed if the percentage is greater than or equal to the quiz's passing score.
8. Save the attempt to attempts.xml with a short unique id and redirect to the grade report page.

FIGURE 3.2 – ATTEMPT FLOW
Start → Timer runs → Student answers → Submit (or auto-submit at 0:00)
→ Evaluate answers → Percentage vs passing score → Save attempt in attempts.xml
→ Grade report displayed

Figure 3.2: Quiz attempt and evaluation flow

3.15 Dashboard Statistics

The student dashboard loads published quizzes and the student's attempts. It counts completed quizzes, counts passed attempts, and calculates the average and the highest percentage. For each quiz it shows the latest attempt so that the card displays New, Passed or Failed. The administrator dashboard similarly shows the number of quizzes, published quizzes, bank questions, students, submissions and the average grade, along with the recent submissions table.

3.16 Timer and Quiz Navigation

quiz-timer.js defines a QuizController. It starts a one-second interval that decreases the remaining time and updates the clock, shows the question count and progress bar, moves between questions through the Next/Previous buttons or the question matrix, highlights answered questions, and gives a visual warning when 60 seconds or less remain. When the time reaches zero, it calls submitQuiz(true) to submit the form automatically.

3.17 URL Design

Servlet	URL patterns	Purpose
AuthServlet	/login, /register, /logout	Authentication and registration.
AdminDashboardServlet	/admin/dashboard	Instructor console with statistics.
AdminQuizServlet	/admin/quizzes, /admin/quiz/new, /edit, /save, /delete, /toggle-publish	Quiz CRUD and publish toggle.
AdminQuestionServlet	/admin/questions, /admin/question/new, /edit, /save, /delete	Question CRUD.
AdminAttemptServlet	/admin/attempts, /admin/attempt/detail	Submissions list and detailed evaluation.
AdminUserServlet	/admin/users, /admin/user/delete	User listing and deletion.
StudentDashboardServlet	/student/dashboard	Student assessment center.
StudentQuizServlet	/student/quiz/start, /submit, /result	Taking and evaluating a quiz.
StudentHistoryServlet	/student/history	Attempt history.

Table 3.5: URL mapping of servlets

3.18 Error Handling

Servlets validate request parameters and redirect with a readable message when a quiz is not found, is not published or has no questions yet. Login and registration errors (for example “Invalid username or password” and “Username already exists”) are shown on the same form. Unexpected exceptions are wrapped in a ServletException so that the container reports them.

3.19 Implementation Plan

The implementation begins with the model classes and XML files. XMLHelper is then written to read and write users, quizzes and attempts. AuthServlet and AuthFilter provide login and role guarding. Administrator servlets and JSP pages are created for quizzes, questions, submissions and users. The student dashboard, quiz session, evaluation and history are added next, followed by the CSS styling and the JavaScript timer. Finally the project is packaged as a WAR file and deployed on Tomcat for testing.

CHAPTER 4
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation Overview

The supplied ZIP contains a Maven project named JavaQuizApp, a ready-built JavaQuizApp.war, an Apache Tomcat 10.1.61 folder and the start.bat, run.bat and stop.bat scripts for one-click start-up and shutdown. The source code consists of 18 Java classes in the filter, model, servlet and util packages, JSP pages under WEB-INF/jsp, one stylesheet and one JavaScript file.

4.2 Authentication Module

The login page (Figure 4.1) asks for a username and password and offers an “Enroll Here” link for new students. The pre-configured accounts are listed on the page for demonstration. After a successful login the user is redirected to the dashboard that matches the role.

Role	Username	Password
Instructor (Admin)	admin	admin123
Student	student1	student123

Table 4.1: Pre-configured accounts

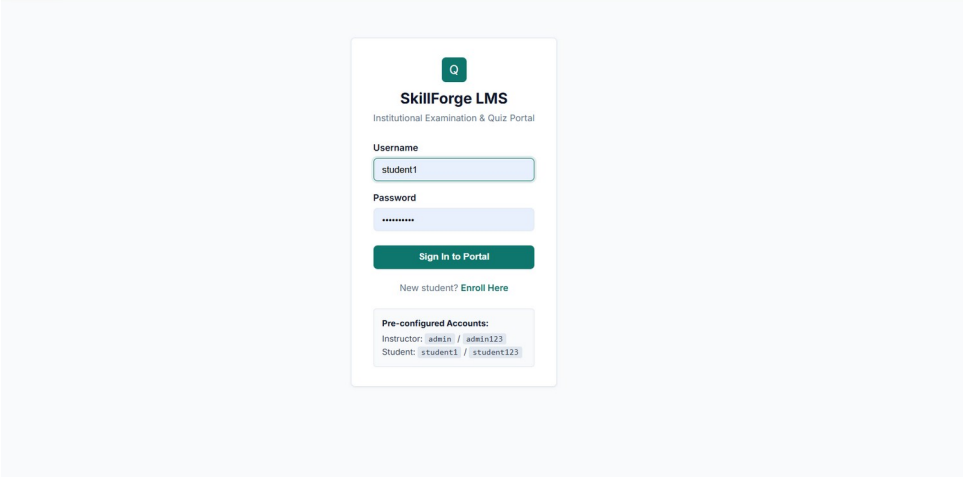


Figure 4.1: Login page

4.3 Student Dashboard Module

StudentDashboardServlet loads the published quizzes and the student’s attempts. The Assessment Center (Figure 4.2) shows four summary cards (Quizzes Completed, Passed Assessments, Average Grade and Highest Grade) and one card per quiz. The “Java Basics” card shows its category (Programming), the number of questions and total marks (3 questions, 30 points), the time allocation (10 min), the passing threshold (60%) and buttons to view results or retake the exam.

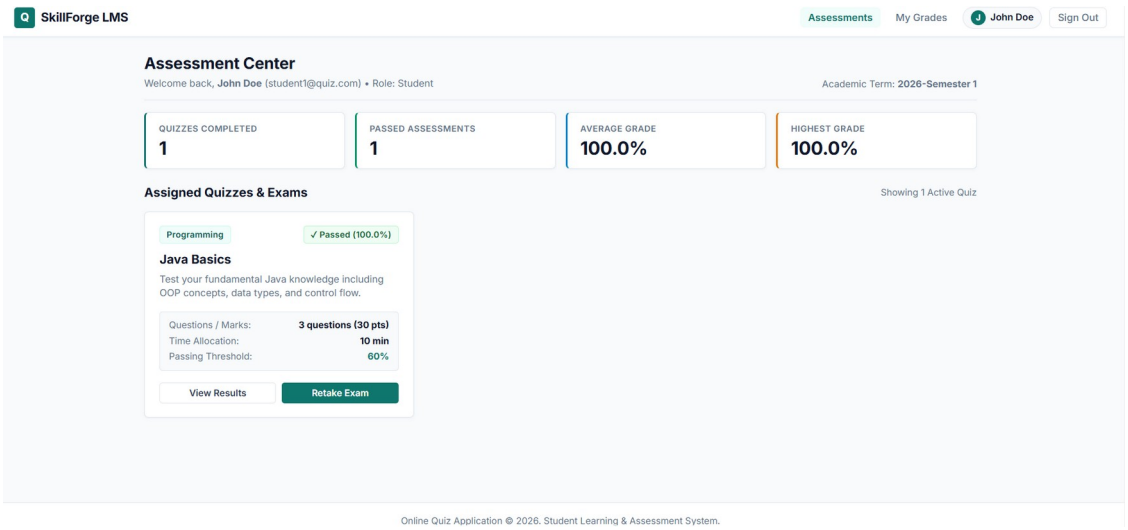


Figure 4.2: Student assessment center (dashboard)

4.4 Quiz Attempt Module

StudentQuizServlet starts the exam session. The page (Figure 4.3) shows the quiz title, “Question 1 of 3”, a progress bar, a live countdown (9:59 at the start of a 10-minute quiz), the four options A to D and a Next Question button. The Exam Navigator on the right shows a question matrix with Answered, Active and Pending states. The exam ends either when the student submits the last question or when the timer expires.

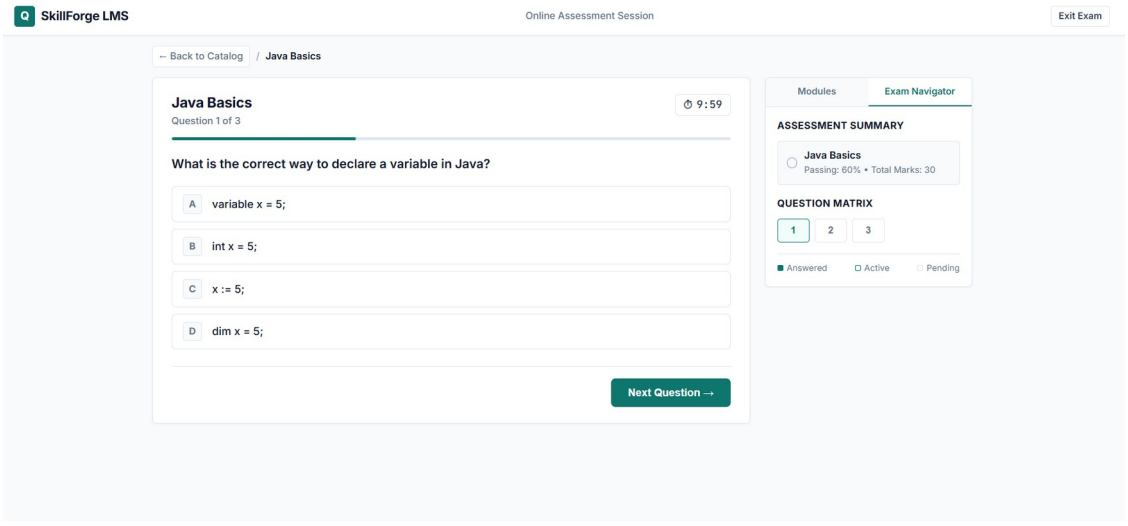


Figure 4.3: Online assessment session

4.5 Evaluation and Grade Report Module

When the quiz is submitted the answers are evaluated as described in Section 3.14. The grade report (Figure 4.4) shows the percentage (100.0%), the Passed badge, the attempt id, the score (30 out of 30 points), the required benchmark (60%), counters for correct, incorrect and unanswered questions, the duration taken, and an item-by-item evaluation. For each question it shows the student’s choice, the correct answer, the points earned and the academic rationale entered by the instructor.

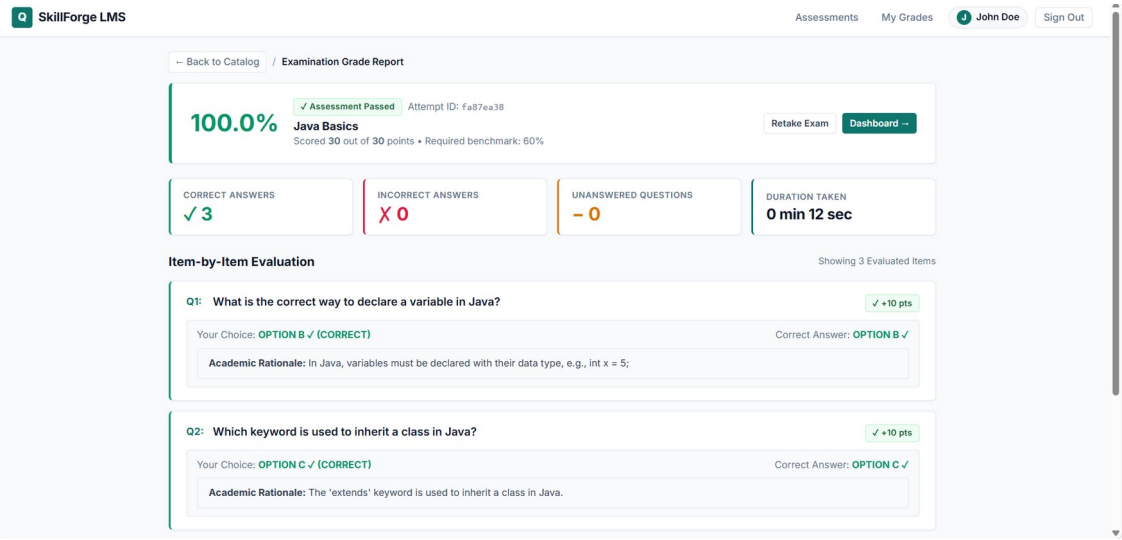


Figure 4.4: Examination grade report

4.6 Administrator Dashboard Module

AdminDashboardServlet provides the Instructor Administration Console (Figure 4.5). It shows cards for Total Quizzes, Published, Bank Questions, Students, Submissions and Average Grade, a Curriculum Assessments table with category, number of questions and marks, timer limit and status, and a Recent Student Examination Submissions table with record id, student name, assessment, score, grade, status and submission time. The record in the screenshot (fa87ea38, John Doe, 30 / 30, 100.0%, Passed) is the same attempt that appears in the student's grade report, which shows that the attempt was saved and is visible to the instructor.

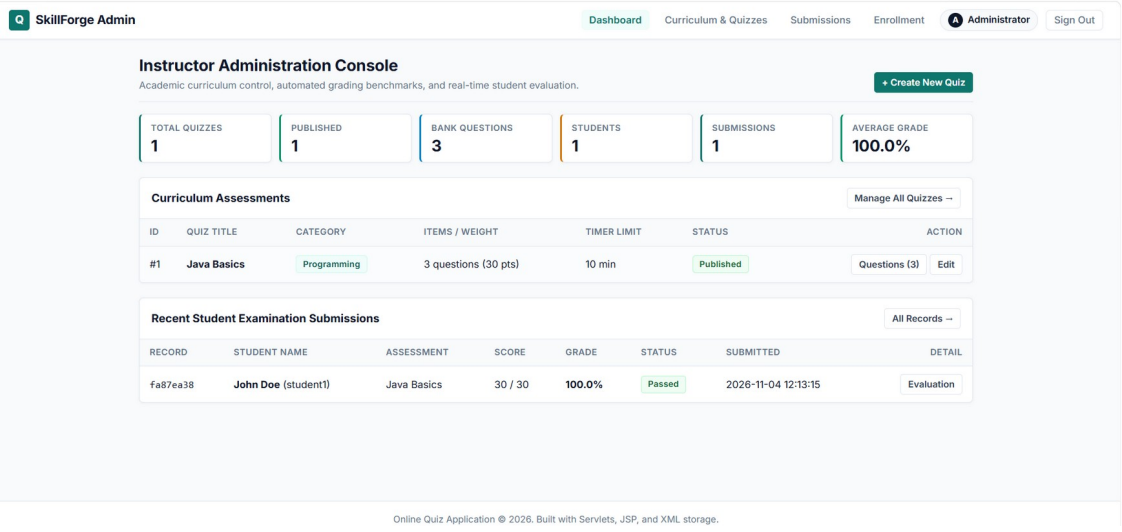


Figure 4.5: Instructor administration console

4.7 Quiz and Question Management Module

AdminQuizServlet and AdminQuestionServlet support the full create, read, update and delete cycle. The "Define New Assessment" form (Figure 4.6) captures the assessment title, syllabus or description, subject category, passing benchmark (default 60%), overall time limit in seconds, time per question in seconds (0 means untimed) and a checkbox to publish the assessment immediately. The "Create & Add Questions" button then takes the instructor to the question screen where each question, its four options, the correct option, marks and explanation are entered.

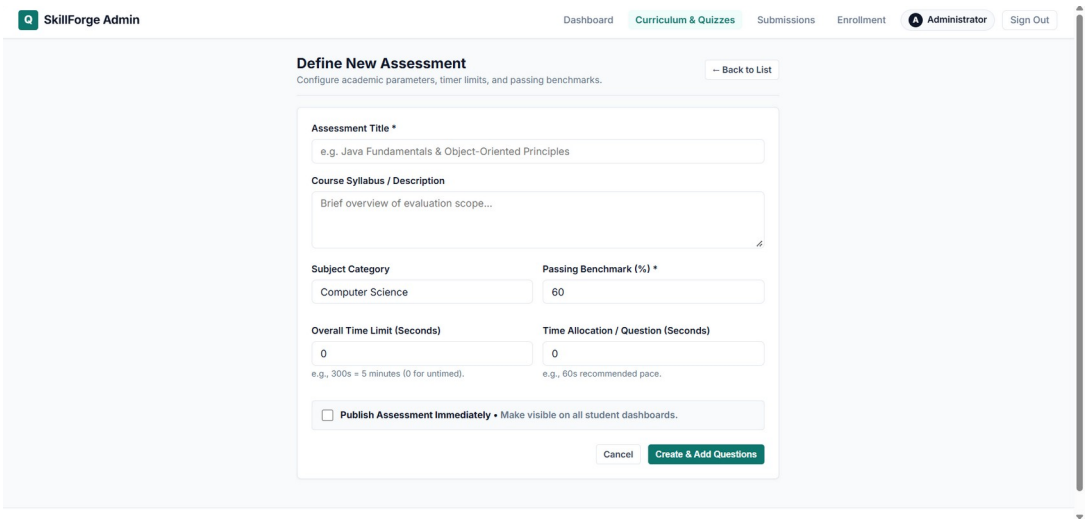


Figure 4.6: Define new assessment form

4.8 Submissions and User Management

AdminAttemptServlet lists all student attempts, can filter them by quiz and opens a detailed view of a single attempt with the answers, right and wrong breakdown and time spent. AdminUserServlet lists registered student and administrator accounts and allows deletion, except of the administrator who is currently logged in.

4.9 Student History Module

StudentHistoryServlet, reached through the “My Grades” link, lists the past attempts of the logged-in student so that performance can be reviewed at any time.

4.10 Frontend Implementation

The interface is built with JSP pages that share header and footer fragments for the student and administrator areas, a single stylesheet (css/style.css) and the quiz-timer.js controller. The design uses a consistent teal colour theme, card-based layouts and responsive containers.

4.11 Data Storage Integration

XMLHelper centralises all XML access. Each public method (for example getAllUsers, getPublishedQuizzes, getQuizById, saveAttempt and getAttemptsByUser) parses the relevant file with DocumentBuilder, converts elements to model objects or the reverse, and writes changes back with the Transformer API. The data directory is obtained from the servlet context as /WEB-INF/data.

4.12 Sample Quiz Data

Item	Value
Quiz title	Java Basics
Category	Programming
Description	Test fundamental Java knowledge including OOP concepts, data types and control flow.
Number of questions	3 (MCQ), 10 marks each, total 30
Total time limit	600 seconds (10 minutes); 60 seconds per question
Passing score	60%

Item	Value
Status	Published

4.13 Validation Approach

Validation is organised around the main user workflows and business rules. The cases marked "Observed" were executed during development and are supported by the screenshots in this chapter; the remaining cases describe the expected behaviour and should be recorded as Pass/Fail by the student after running the application.

Test ID	Test scenario	Expected result	Status
TC-01	Log in as student1 / student123	Student dashboard (Assessment Center) opens.	Observed (Fig. 4.1, 4.2)
TC-02	Register using an existing username	"Username already exists" message is shown.	To be recorded
TC-03	Student opens an /admin/* URL	User is redirected to the student dashboard.	To be recorded
TC-04	Open a published quiz	Exam page with timer 9:59 and question 1 of 3 opens.	Observed (Fig. 4.3)
TC-05	Answer all three questions correctly and submit	30 / 30, 100.0%, status Passed.	Observed (Fig. 4.4)
TC-06	Let the timer reach zero	Quiz is submitted automatically.	To be recorded
TC-07	Answer incorrectly or skip questions	Marks are reduced; incorrect/unanswered counters increase; Failed if below 60%.	To be recorded
TC-08	Dashboard statistics after one attempt	Completed 1, Passed 1, Average 100.0%, Highest 100.0%.	Observed (Fig. 4.2)
TC-09	Log in as admin / admin123	Instructor console opens with statistics.	Observed (Fig. 4.5)
TC-10	Check the admin submissions table	The student's attempt appears with score and status.	Observed (Fig. 4.5)
TC-11	Open the Define New Assessment form	Form with default passing benchmark 60 opens.	Observed (Fig. 4.6)
TC-12	Create a quiz, add a question and publish it	Quiz appears on the student dashboard.	To be recorded

Table 4.2: Test cases and outcomes

4.14 Data Validation

Servlets check that the requested quiz or attempt exists before using it, that a quiz is published and contains questions before it can be started, and that a username is not already registered. Marks are always calculated on the server from the stored correct option, not from any value sent by the browser.

4.15 Result Interpretation

The most important functional result is the correctness of the evaluation. In the demonstration run the student selected Option B for Question 1 and Option C for Question 2, both matching the stored correct options, and the report shows 3 correct, 0 incorrect and 0 unanswered, giving

30 out of 30 marks (100.0%) in 0 min 12 sec. Because 100% is above the 60% benchmark, the attempt was marked Passed and the same figures appear on the student's dashboard and the administrator's console.

4.16 Limitations in Validation

The project contains application source code and documentation but no automated unit tests and no measured performance results. Only a single student account and a single quiz were used in the demonstration, so behaviour with many simultaneous users has not been measured.

4.17 Project Management

The project was developed by two students. The work was divided into requirement analysis, data and model design, servlet and JSP development, timer scripting, testing and documentation, following the timeline in Section 1.11.

Team member	Contribution area
Arnav Mittal (24BCS10123)	Requirement analysis, backend (Servlet/JSP and XML) implementation, integration and report compilation.
Rajan (24BCS12890)	Frontend interface review, testing, validation scenarios and user manual.

The contribution areas above are a suggested report presentation structure. The students should edit this table to reflect the actual work performed by each member.

4.18 Security and Reliability Review

- AuthFilter protects the administrator and student areas and redirects unauthorised users.
- JSP files are placed under WEB-INF so they cannot be requested directly; the session cookie is HttpOnly.
- Marks are calculated on the server so students cannot alter their score from the browser.
- Passwords are stored and compared as plain text in users.xml. A production version should store salted hashes using BCrypt or Argon2.
- The time limit is enforced mainly by the browser script; a production system should also check the elapsed time on the server before accepting a submission.
- The XML files have no transaction or locking support, so a production system should use a database with proper concurrency control.
- A production version should add HTTPS, CSRF protection and input escaping review.

4.19 Overall Analysis

The project demonstrates an end-to-end examination workflow with role-based access, timed attempts, automatic evaluation and result review. Its main academic strengths are the clean separation of filter, servlets, data helper and views, the use of XML for zero-configuration persistence, and the explanation-based feedback given to the student.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Online Quiz Application provides a web-based demonstration of the essential activities of an online examination system. It combines authentication, quiz and question management, publishing, a timed quiz session, automatic evaluation, grade reports with explanations, attempt history and an administrator console in one application. Java Servlets and JSP are used for the backend and views, while XML files provide lightweight persistence.

The project shows how a layered structure can organise a multi-feature web application. The filter guards access, servlets handle requests, XMLHelper manages data and JSP pages present the results. The project therefore provides practical experience in Java web development, XML processing, session management, client-side scripting and software documentation.

5.2 Deviation from Expected Results

The supplied project is demonstrated with a small sample dataset of one quiz with three questions and one student account. This is useful for demonstration but differs from real institutional examinations, which involve large question banks, many candidates and stricter security. No measured performance benchmark or automated test report was included.

5.3 Limitations

- Only multiple-choice questions with a single correct answer are supported.
- Passwords are stored as plain text.
- The time limit is enforced mainly on the client side.
- XML file storage does not provide transactions, indexing or safe concurrent writes.
- No e-mail notification, proctoring or anti-cheating feature is included.
- Question randomisation and question banks shared between quizzes are not implemented.

5.4 Future Work

1. Replace plain-text passwords with salted BCrypt or Argon2 hashes.
2. Replace XML storage with a relational database such as MySQL using JDBC or JPA.
3. Enforce the time limit on the server and randomise question and option order.
4. Add more question types such as multiple-select, true/false and short answer.
5. Add charts and analytics for instructors, such as question-wise difficulty.
6. Add e-mail notifications and downloadable result reports in PDF.
7. Add password reset, profile management and class or batch grouping.
8. Add automated unit and integration tests.
9. Deploy to a secure cloud environment with HTTPS and backups.
10. Improve accessibility and multilingual support.

5.5 Final Remarks

The project meets its academic purpose by demonstrating a complete quiz and examination workflow using core Java web technologies. With additional security, storage, testing and deployment work, the prototype could be extended into a more realistic assessment platform.

REFERENCES

1. Oracle. (2024). Java Platform, Standard Edition Documentation. Oracle.
2. Eclipse Foundation. (2024). Jakarta Servlet Specification 5.0. Eclipse Foundation.
3. Eclipse Foundation. (2024). Jakarta Server Pages Specification. Eclipse Foundation.
4. Apache Software Foundation. (2024). Apache Tomcat 10.1 Documentation. Apache.
5. Apache Software Foundation. (2024). Apache Maven Documentation. Apache.
6. Oracle. (2024). Java API for XML Processing (JAXP) Tutorial. Oracle.
7. W3C. (2008). Extensible Markup Language (XML) 1.0. World Wide Web Consortium.
8. WHATWG. (2024). HTML Living Standard. Web Hypertext Application Technology Working Group.
9. MDN Web Docs. (2024). JavaScript Guide. Mozilla.
10. Project source code and README supplied in Java_Quiz.zip. (2026).

APPENDIX A

USER MANUAL

A.1 System Requirements

- A computer running Windows (the supplied scripts are .bat files); Linux or macOS can run Tomcat manually.
- Java Development Kit (JDK) 11 or later.
- Apache Tomcat 10.1 (supplied inside the project folder).
- A web browser such as Chrome, Edge or Firefox.
- Apache Maven only if the WAR file is to be rebuilt.

A.2 Run the Application

1. Extract the project folder “Java Quiz”.
2. Double-click start.bat (or run.bat). It prepares Tomcat, deploys JavaQuizApp.war, starts the server and opens the browser.
3. If the browser does not open, visit http://localhost:8080/JavaQuizApp/ manually.
4. Sign in using the pre-configured accounts in Table 4.1, or register a new student account.
5. When finished, double-click stop.bat to stop the server.

A.3 Rebuild the WAR File

Open a terminal in the JavaQuizApp folder and run build.bat (Command Prompt), build.ps1 (PowerShell) or mvn clean package. The WAR file is created in the target folder and can be copied to Tomcat’s webapps folder. Then run bin\startup.bat.

A.4 Main User Workflow

Registration and Login

Open the application and choose “Enroll Here” to create a student account. Sign in with the username and password; you are taken to the Assessment Center.

Take a Quiz

Click the quiz card’s start button. Select one option for each question, move with Next Question or the question matrix, and submit on the last question. The quiz is submitted automatically if the timer reaches zero.

View Results

After submission the grade report opens. Use “View Results” on the dashboard or “My Grades” to see it again, and “Retake Exam” to try the quiz again.

Create a Quiz (Instructor)

Sign in as admin, open Curriculum & Quizzes, choose “Create New Quiz”, fill in the assessment details and click “Create & Add Questions”. Add each question with four options, mark the correct one, enter marks and an explanation. Tick “Publish Assessment Immediately” or use the publish toggle later.

Review Submissions (Instructor)

Open Submissions to see all attempts, filter by quiz and open the Evaluation page for a detailed breakdown. Use Enrollment to view or delete user accounts.

A.5 Demonstration Scenario

1. Start the server and sign in as student1 / student123.
2. Open the Java Basics quiz and answer the three questions.
3. Submit and observe the grade report.
4. Sign out and sign in as admin / admin123.
5. Open the dashboard and verify that the submission appears in the recent submissions table.

A.6 Troubleshooting

Problem	Possible action
Application does not start	Check that Java is installed (java -version) and read the Tomcat logs folder.
Port 8080 is already in use	Stop the other application or run stop.bat, then start again; or change the Tomcat connector port in conf/server.xml.
Browser cannot open the page	Confirm Tomcat has started and use http://localhost:8080/JavaQuizApp/.
Login fails	Check the username and password in Table 4.1 or in WEB-INF/data/users.xml.
Changes are not visible	Rebuild and redeploy the WAR, restart Tomcat and refresh the browser.

APPENDIX B

PROJECT STRUCTURE AND ACHIEVEMENTS

B.1 Project Folder Structure

```
Java Quiz/
├─ start.bat, run.bat, stop.bat, README.md, JavaQuizApp.war
├─ tomcat/apache-tomcat-10.1.61/
└─ JavaQuizApp/
   ├─ pom.xml, build.bat, build.ps1
   └─ src/main/
      ├─ java/com/quiz/ (filter, model, servlet, util)
      ├─ resources/data/ (users.xml, quizzes.xml, attempts.xml)
      └─ webapp/ (index.jsp, css/, js/, WEB-INF/web.xml, WEB-INF/jsp/, WEB-INF/data/)
```

B.2 Important Project Files

File	Purpose
AuthFilter.java	Role-based guard for /admin/* and /student/*.
AuthServlet.java	Login, registration and logout.
StudentQuizServlet.java	Starts a quiz, evaluates answers and shows the result.
AdminQuizServlet.java / AdminQuestionServlet.java	Quiz and question management.
XMLHelper.java	Reads and writes users, quizzes and attempts in XML.
AppContextListener.java	Creates the data files at application start-up.
quiz-timer.js	Countdown timer, navigation and auto-submit.
style.css	Application appearance and responsive styling.
web.xml	Welcome file and session configuration.
pom.xml	Maven metadata, dependencies and WAR packaging.

B.3 Learning Outcomes

- Understanding the development of a Java-based web application.
- Using Servlets, filters and sessions for request handling and access control.
- Creating JSP pages and forwarding data from servlets.
- Reading and writing XML using the Java DOM and Transformer APIs.
- Using JavaScript for a countdown timer and question navigation.
- Implementing automatic evaluation logic and result analysis.
- Preparing test cases and documenting a software project.

B.4 Achievements

The project brings together the core modules of an online examination workflow in a single application. It includes authentication, quiz and question management, a timed attempt session, automatic evaluation, grade reports with explanations, student history and an administrator

console. It also provides a structured codebase that can be started with a single script and rebuilt using Maven.

Any external awards, competition results, certificates or formal project evaluation marks should be added here only if they are directly related to this project and supported by evidence.

B.5 Declaration by Students

We declare that this project report has been prepared for academic submission based on the Online Quiz Application project developed by our team. The report should be reviewed by all team members and the project supervisor before final submission.

Arnav Mittal (24BCS10123)
Signature: _____

Rajan (24BCS12890)
Signature: _____